Tipi di dato derivati: `struct`

Salvatore Filippone
salvatore.filippone@uniroma2.it

Federica Montesano
federica.montesano@uniroma2.it



Indice dei Contenuti

1 Introduzione alle Strutture

2 Layout di Memoria

3 Introduzione alle `union`

4 Introduzione alle `enum`

Roadmap

1 Introduzione alle Strutture

2 Layout di Memoria

3 Introduzione alle `union`

4 Introduzione alle `enum`

Definizione di Struttura (struct) e typedef

Dichiarazione e typedef

```c
// Definizione con tag
struct Libro {
    char titolo[30];
    char autore[30];
    int anno;
    float prezzo;
};

struct Libro libro1;

// Uso di typedef per semplificare il tipo
typedef struct Libro Libro;
Libro libro2; // Nessun bisogno di 'struct Libro'
```

Cos'è una struct?

Tipo di dato derivato, collezione di dati di tipi potenzialmente disomogenei, raggruppate sotto un unico nome in un blocco contiguo di memoria.

Vantaggi del typedef

Permette di definire un alias di tipo, evitando di dover ripetere la parola chiave `struct` in ogni dichiarazione.

Inizializzazione e Accesso ai Membri

```c
typedef struct {
    char name[20];
    int age;
    float weight;
    int height; // in centimeters
} Person;

// 1. Init
Person p1 = {"Mario", 30, 70.5, 178};
Person p2 = {.age = 25, .name = "Luigi"};

// 2. Direct Access (.)
p1.age += 1;

// 3. Pointer Access (->)
Person *ptr = &p1;
ptr->age = 32;        // equivalent to (*ptr).age
```

Operatori di Accesso

- **Operatore Punto (.):** Usato per accedere ai membri di una variabile struttura diretta.
- **Operatore Freccia (->):** Usato quando si lavora con un **puntatore a struttura**.

`ptr->campo` $\iff$ `(*ptr).campo`

Strutture Autoriferite (Self-Referential)

```c
// Nodo di una lista concatenata
typedef struct Node {
    int data;
    struct Node *next; // Puntatore a se stesso!
} Node;

int main() {
    Node *head = malloc(sizeof(Node));
    head->data = 10;

    head->next = malloc(sizeof(Node));
    head->next->data = 20;
    head->next->next = NULL;

    // Liberazione memoria...
}
```

Strutture Dati Dinamiche

Una struttura è **autoriferita** se contiene un membro che è un puntatore ad una struttura del suo stesso tipo.

Applicazioni

Essenziale per realizzare strutture dati dinamiche avanzate:

- Liste concatenate
- Alberi binari
- Grafi e Code/Pile

Roadmap

1 Introduzione alle Strutture

2 Layout di Memoria

3 Introduzione alle `union`

4 Introduzione alle `enum`

Allineamento in Memoria e Struct Padding

```c
struct Esempio {
    char a;

    int b;
    char c;

};

printf("%lu\n", sizeof(struct Esempio));
// Cosa restituisce?
```

Cos'è lo Struct Padding?

Il compilatore inserisce byte vuoti (*padding*) tra i membri per allineare le variabili agli indirizzi di memoria nativi dell'architettura (es. word di 4 o 8 byte).

Ottimizzazione

Per ridurre lo spreco di memoria, ordinare i membri della struttura dal **più grande al più piccolo**.

Array di Strutture

Array di Strutture

```
Person class[30];

class[0].age = 20;
strcpy(class[0].name, "Alice");

// Accesso al secondo elemento via puntatore
Person *p = &class[1];
p->age = 21;
```

Ruolo nell'Ingegneria Software

Le strutture sono fondamentali per:

- Astrazione dei dati: Creare Abstract Data Types (ADT).
- Incapsulamento: Raggruppare stato e informazioni correlate.
- Manutenibilità: Facilitare l'estensione del codice senza romperne l'interfaccia.

Roadmap

1 Introduzione alle Strutture

2 Layout di Memoria

3 Introduzione alle `union`

4 Introduzione alle `enum`

Unioni (union) in C

```c
union Dato {
    int i;
    float f;
    char str[20];
};

union Dato d;
d.i = 10;
printf("d.i: %d\n", d.i);

d.f = 220.5; // Sovrascrive d.i!
printf("d.f: %.1f\n", d.f);

// Dimensione = membro piu' grande
printf("sizeof: %lu\n", sizeof(d)); // 20 byte
```

Cos'e una union?

Tipo derivato simile alla `struct`, ma i cui membri condividono tutti la **stessa posizione di memoria**.

Caratteristiche Principali

- **Spazio Unico:** Contiene un solo valore valido alla volta.
- **Dimensione:** Pari alla dimensione del membro piu grande (non alla somma).
- **Scopo:** Risparmiare memoria o rappresentare dati varianti.

Roadmap

1 Introduzione alle Strutture

2 Layout di Memoria

3 Introduzione alle `union`

4 Introduzione alle `enum`

Enumerazioni (enum) in C

Cos'e una enum?

Tipo definito dall'utente che permette di assegnare nomi simbolici a costanti intere.

```c
// Valori automatici: LUN=0, MAR=1, MER=2...
enum Giorno { LUN, MAR, MER, GIO, VEN, SAB, DOM };

// Valori interi personalizzati
enum Stato { IDLE = 0, RUNNING = 1, ERROR = -1 };

enum Giorno oggi = MER;

if (oggi == MER) {
    printf("Giorno numero %d\n", oggi); // Output: 2
}
```

Caratteristiche Principali

- **Valori Automatici:** Di default partono da 0 e aumentano di 1.
- **Valori Espliciti:** È possibile assegnare interi specifici a ciascun nome.
- **Vantaggi:** Aumenta la leggibilita del codice.